

Picos Triples

La Cordillera Oriental es una cadena montañosa de los Andes que se extiende a través de Bolivia. Consiste en una secuencia de N picos de montaña, enumeradas desde el 0 hasta $N - 1$. La **altura** del pico i ($0 \leq i < N$) es $H[i]$, un entero entre 1 y $N - 1$ inclusive.

Para cada par de picos i y j , con $0 \leq i < j < N$, la **distancia** entre ellos es definida como $d(i, j) = j - i$.

Acorde a las antiguas leyendas Incas, una terna de picos es **mítica** si satisface la siguiente propiedad especial: las alturas de los tres picos **coinciden** con sus distancias entre ellos mismos, **ignorando el orden**.

Dicho formalmente, una terna de índices (i, j, k) es mítica si

- $0 \leq i < j < k < N$, y
- La terna de alturas $(H[i], H[j], H[k])$ coincide con la terna de distancias $(d(i, j), d(i, k), d(j, k))$ en un cierto orden. Por ejemplo, para los índices $(0, 1, 2)$, la terna de distancias es $(1, 2, 1)$. Con esta indexación, las tres ternas de alturas $(H[0], H[1], H[2]) = (1, 1, 2)$, $(H[0], H[1], H[2]) = (1, 2, 1)$ y $(H[0], H[1], H[2]) = (2, 1, 1)$ coinciden con la misma terna de distancia, pero las alturas $(H[0], H[1], H[2]) = (1, 2, 2)$, no.

Este problema consiste de dos partes, con cada subtask asociada a la **Parte I** o **Parte II**. Puedes resolver las subtasks en cualquier orden. En particular, **no necesitas** completar toda la Parte I antes de intentar la Parte II.

Parte I

Dada una descripción de una cordillera (cadena de montañas), tu trabajo es contar la cantidad de ternas míticas.

Detalles de la Implementación

La implementación construir la siguiente función.

```
long long count_triples(std::vector<int> H)
```

- H : arreglo de longitud N , representando las alturas de los picos.
- Esta función es llamada exactamente una vez por cada test case.

Esta función deberá devolver un entero T , la cantidad de ternas míticas en la cordillera.

Restricciones

- $3 \leq N \leq 200\,000$
- $1 \leq H[i] \leq N - 1$ para cada i tal que $0 \leq i < N$.

Subtareas

Parte I equivale a un total de 70 puntos.

Subtarea	Puntos	Restricciones adicionales
1	8	$N \leq 100$
2	6	$H[i] \leq 10$ para cada i tal que $0 \leq i < N$.
3	10	$N \leq 2000$
4	11	Las alturas son no decrecientes. Esto es, $H[i - 1] \leq H[i]$ para cada i tal que $1 \leq i < N$.
5	16	$N \leq 50\,000$
6	19	Sin restricciones adicionales.

Ejemplo

Considere la siguiente llamada.

```
count_triples([4, 1, 4, 3, 2, 6, 1])
```

Hay 3 ternas míticas en esta cordillera:

- Para $(i, j, k) = (1, 3, 4)$, las alturas $(1, 3, 2)$ coinciden con sus respectivas distancias $(2, 3, 1)$.
- Para $(i, j, k) = (2, 3, 6)$, las alturas $(4, 3, 1)$ coinciden con sus respectivas distancias $(1, 4, 3)$.
- Para $(i, j, k) = (3, 4, 6)$, las alturas $(3, 2, 1)$ coinciden con sus respectivas distancias $(1, 3, 2)$.

Por ello, la función debería devolver 3.

Note que los índices $(0, 2, 4)$ no forman una terna mítica, ya que las alturas $(4, 4, 2)$ no coinciden con las distancias $(2, 4, 2)$.

Parte II

Tu trabajo es construir una cordillera con una cierta cantidad de ternas míticas. Esta parte consiste en 6 subtareas de **solo-output** con posibilidad a **puntaje parcial**

En cada subtarea, recibes dos enteros positivos M y K , y deberás construir una cordillera con **a lo mucho** M picos. Si tu solución contiene **al menos** K ternas míticas, recibirás el puntaje completo por esta subtarea. Caso contrario, tu puntaje será proporcional a la cantidad de ternas míticas que contenga tu solución.

Observa que tu solución debe consistir de una cordillera válida. Específicamente, supón que tu solución tiene N picos (N debe satisfacer $3 \leq N \leq M$). También debe satisfacer que la altura de cada pico i ($0 \leq i < N$), denotada por $H[i]$, debe ser un entero entre 1 y $N - 1$, inclusive

Detalles de la Implementación

Hay dos formas de enviar tu solución, y puedes usar cualquiera de ellas para cada subtarea:

- **Output file**
- **Procedure call**

Para enviar tu solución tipo **output file**, crea y envía un archivo de texto con el siguiente formato:

```
N
H[0] H[1] ... H[N-1]
```

Para enviar tu solución tipo **procedure call**, deberás implementar la siguiente función.

```
std::vector<int> construct_range(int M, int K)
```

- M : la máxima cantidad de picos
- K : El número deseado de ternas míticas
- Esta función es llamada exactamente una vez por subtarea

La función deberá devolver un arreglo H de longitud N , representando las alturas de los picos.

Subtareas y Puntuación

Parte II equivale un total de 30 puntos. Para cada subtarea, los valores de M y K son fijos y siguen la siguiente tabla.

Subtarea	Puntaje	M	K
7	5	20	30
8	5	500	2000
9	5	5000	50 000
10	5	30 000	700 000
11	5	100 000	2 000 000
12	5	200 000	12 000 000

Para cada subtarea, si tu solución es una montaña válida, tu puntaje será 0 puntos (reportado como `Output isn't correct` en CMS).

Por otro lado, sea T la cantidad de ternas míticas en tu solución. En ese caso, el puntaje de tu subtarea es:

$$5 \cdot \min \left(1, \frac{T}{K} \right).$$

Grader de Ejemplo

Las partes I y II siguen el mismo criterio del grader, con una distinción entre las dos partes determinada por la primera línea del input.

Formato del Input para la Parte I:

```
1
N
H[0] H[1] ... H[N-1]
```

Formato del Output para la Parte I:

```
T
```

Formato del Input para la Parte II:

```
2
M K
```

Formato del Output para la Parte II:

```
N
```

```
H[0]  H[1]  ... H[N-1]
```

Observe que el output del grader coincide con el formato requerido para el archivo output de la Parte 2.